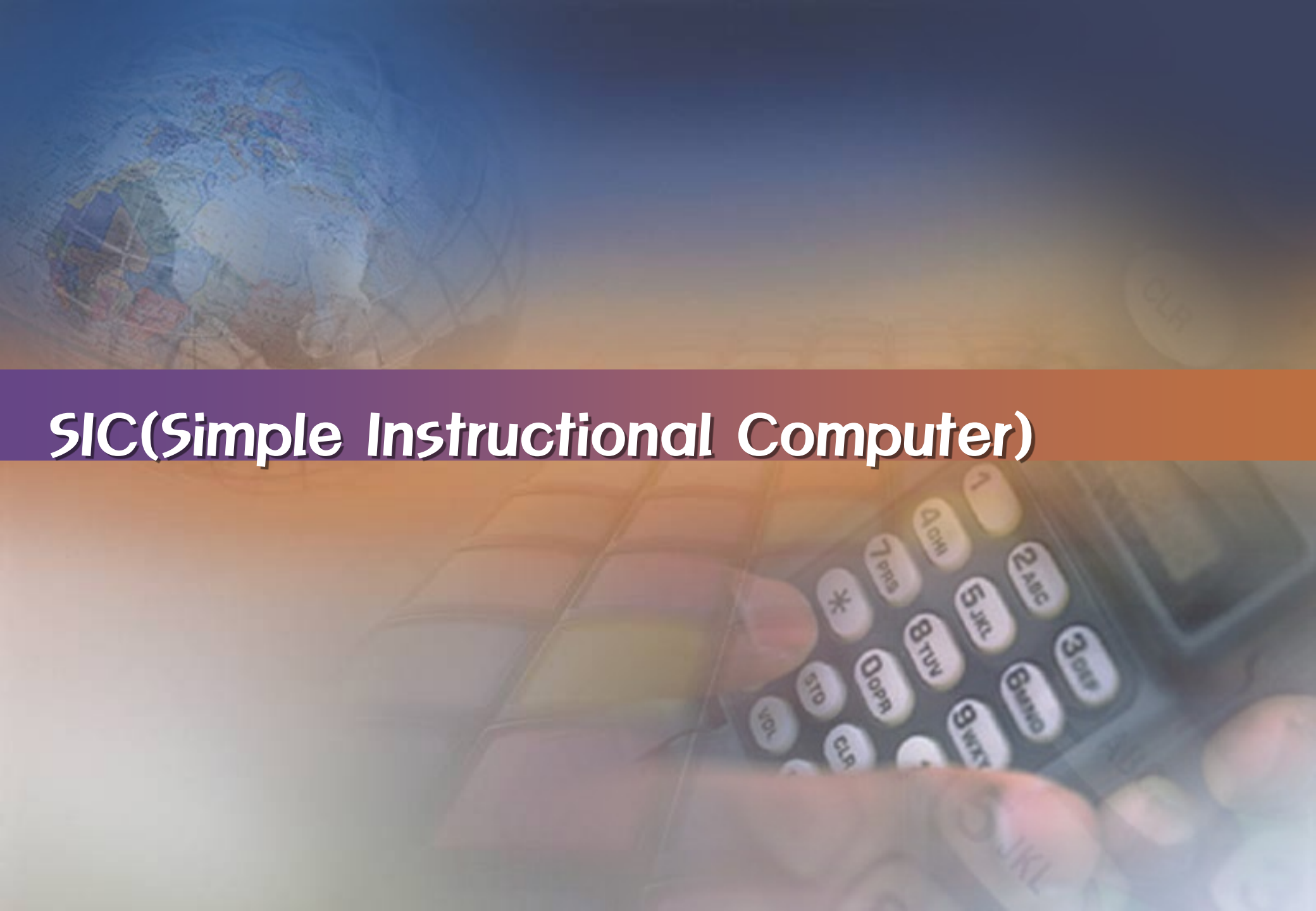




SIC(Simple Instructional Computer)

수업 목표

- SIC(Simplified Instructional Computer)의 개념을 이해한다.
- SIC 명령어 구조와 사용방법에 대해 학습한다.
- 시스템 소프트웨어와 컴퓨터 시스템과의 관계를 이해한다.



SIC(/XE) Machine Architecture

- SIC (Simplified Instructional Computer)

- Most commonly encountered H/W features and concepts, while avoiding most of the idiosyncrasies that are often found in real machines

- SIC/XE

- XE version of SIC Machine
 - Extra Equipment
 - Extra Expensive

*실제 컴퓨터 명령어를 단순화시켜 이해
하기 쉽게 만들어 놓은 가상의 컴퓨터*

- SIC & SIC/XE

- Upward compatible

Memory

- Structure

- 1 byte - 8 bit
- 1 word - 3 bytes (24 bits)
- All addresses on SIC are byte address
 - Words are addressed by lowest numbered byte
- Max memory size - 32,768 (2^{15}) bytes

- SIC은 8비트가 한 byte로 이루어져 있고 모든 주소는 byte로 표현되며 저장
- 3bytes가 한 word로 이루어져 있으며 SIC의 전체 메모리는 2의 15승이다.
- SIC은 가상 컴퓨터의 standard 모델이고 SIC/XE는 확장된 모델이다.

Registers

- 5 registers

- all have their own special purpose
- 24-bit length

- *Register A*는 산술연산에 사용
- *Register X*는 주소 지정에 사용
- *Register L*은 부루틴 호출 후 복귀 주소 저장에 사용
- *Register PC*는 실행할 다음 명령 주소 저장에 사용
- *Register SW*는 조건코드와 같은 다양한 정보 저장에 사용

- Register name

Mnemonic	Number	Special Use
A	0	A ccumulator; for arithmetic operations
X	1	Index x register; for addressing
L	2	L inkage register; store return address
PC	8	P rogram C ounter; contain the address of the next instruction
SW	9	S tatus W ord; contain a variety of info, including a Condition Code (CC)

Data Formats

- Integer: 24-bit binary number

- 2's complement representation

- $-2^{23} \sim 2^{23} - 1$

음수는 2의 보수로 나타내며, 2의
보수는 1의 보수 +1로 저장

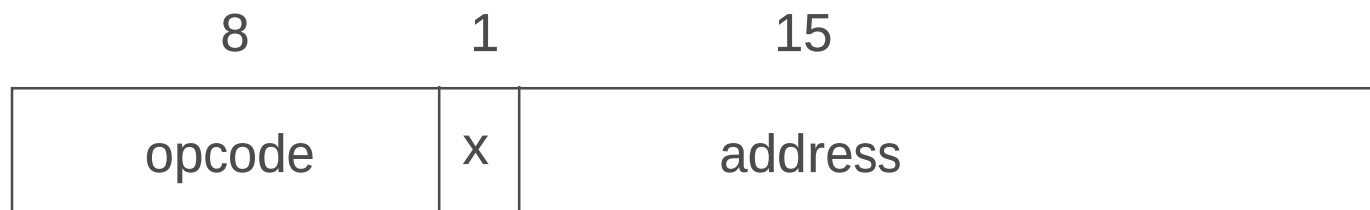
- Character: 8-bit ASCII code

- See Appendix B

- No floating-point H/W on SIC

Instruction Formats

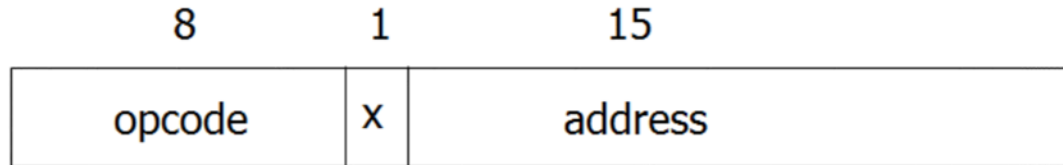
- All instructions on SIC have 24-bit format (3 bytes, 1 word)
 - Opcode – First 8 bits
 - Index Code – Next bit
 - Address – Last 15 bits



x: indicate indexed-addressing mode

x 비트가 1로 셋팅되어 있는 것은 SIC 명령어가 인덱스 주소 지정 방식을 사용한다는 것을 나타냄

Addressing Modes



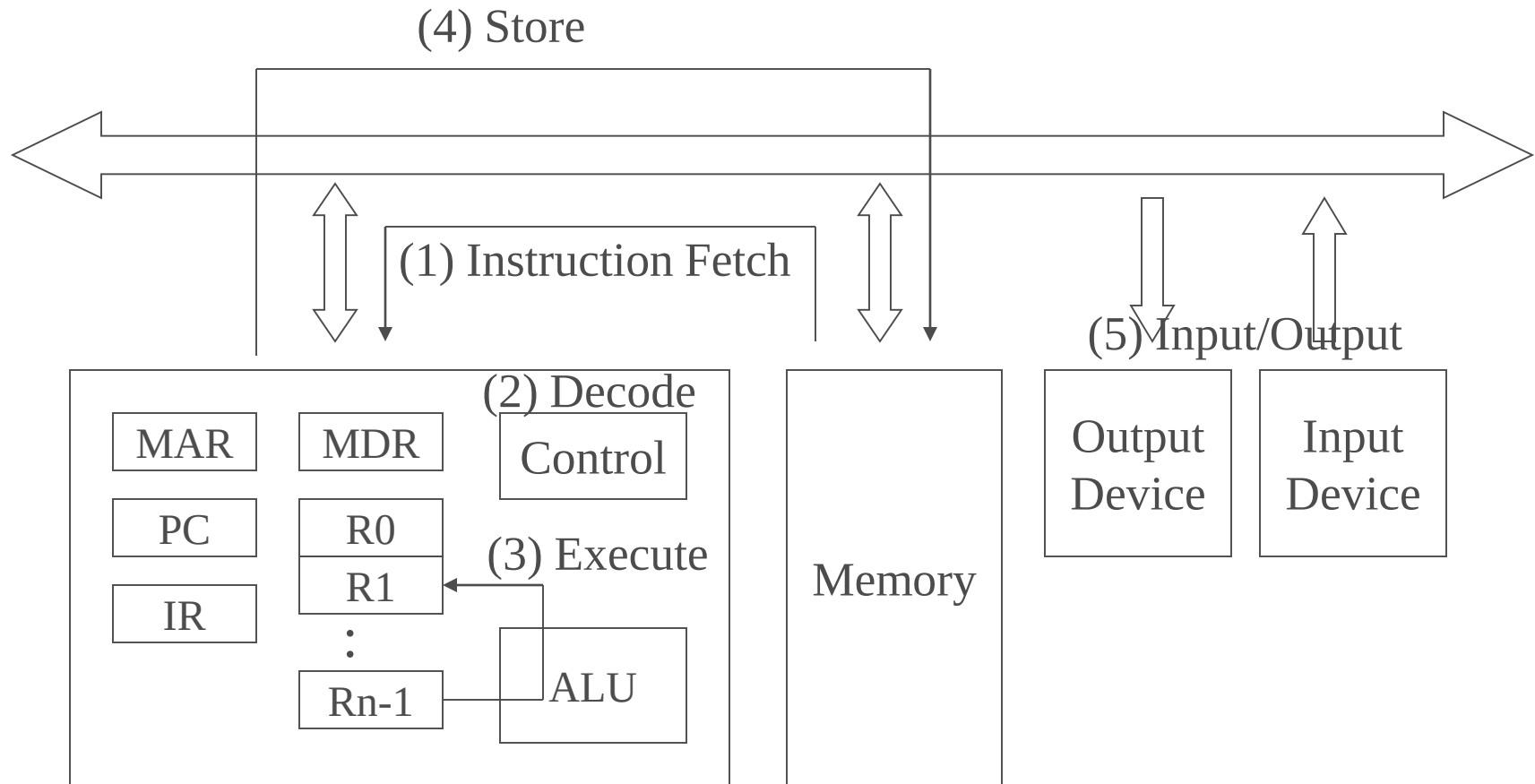
x: indicate indexed-addressing mode

Mode	Indication	Target address calculation
Direct	$x = 0$	$TA = \text{address}$
Indexed	$x = 1$	$TA = \text{address} + (X)$

TA : Target Address

(X) : the contents of register X

Basic Structure of the Computer System



Instruction Set (SIC)

- **Load & Store**
 - (1) LDA, LDX, LDL, LDCH
 - (2) STA, STX, STL, STCH, STSW
- **Arithmetic & Logical**
 - (3) ADD, SUB, MUL, DIV, AND, OR, TIX
- **Compare**
 - (4) COMP – compare the value in **A** with a word in memory
& set a condition code CC to indicate the result(<, =, >)
- **Control**
 - (4) (J), JLT, JEQ, JGT – conditional jump
 - (4) JSUB, RSUB – jump to/return from subroutine in **L**
- **I/O**
 - (5) TD, RD, WD

See Appendix A for a complete list of SIC instructions

Instruction Set – Example 1

- SIC Instruction – Load & Store

LDA	ALPHA	A register <- value of ALPHA
LDX	BETA	X register <- value of BETA
STA	GAMMA	GAMMA <- value of A register
STX	DELTA	DELTA <- value of X register

LDCH	ALPHA	A register's value(rightmost byte) <- value of ALPHA
LDL	BETA	L register <- value of BETA
STCH	GAMMA	GAMMA(byte) <- A register's value(rightmost byte)
STL	DELTA	DELTA <- value of L register
STSW	EPSILON	EPSILON <- value of SW register

Instruction Set – Example 2

- SIC Instruction – Arithmetic & Logic

ADD	ALPHA	A register $\leftarrow$ A register + ALPHA
SUB	BETA	A register $\leftarrow$ A register - BETA
MUL	GAMMA	A register $\leftarrow$ A register x GAMMA
DIV	DELTA	A register $\leftarrow$ A register / DELTA
AND	EPSILON	A register $\leftarrow$ A register & EPSILON
OR	ZETA	A register $\leftarrow$ A register ZETA

• COMP ALPHA

$\Leftrightarrow$ CC(of SW) $\leftarrow$ (< , = , >) of (value of A register)?(ALPHA)

※ TIX m

$x \leftarrow (X) + 1; (X) : (m..M+2)$

Instruction Set – Example 3

- SIC Instruction – Conditional Jump

- Test the setting CC(of SW), and jump accordingly

J	ALPHA	PC <- ALPHA
JLT	BETA	PC <- BETA if CC set to <
JGT	GAMMA	PC <- GAMMA if CC set to >
JEQ	DELTA	PC <- DELTA if CC set to =

J는 CC와 상관없이 PC에 ALPHA를 넣음

- SIC Instruction – Subroutine Linkage

JSUB	READ	L <- (PC) (for return) and PC <- READ
RSUB		PC <- (L) (return;)

Set the value of L register & Subroutine call (READ)

Return Subroutine
– Reference the value (PC) of L register

Input and Output

- Transfer **one byte** at a time to/from **the rightmost 8 bits of A**
 - **TD** (Test Device) tests whether the addressed device ready to send/receive **a byte** of data.

The result of this test is:

- < means ready to send/receive
- = means device busy
- > means device not operational

- **RD** (Read Data) m

I/O는 한번에 1바이트씩 전송되며, A 레지스터의 마지막 1byte(8bits)를 사용
모든 디바이스는 각각의 구별을 위해 **unique code(8 bits)**가 부여되며,
최대 2⁸개의 디바이스가 붙을 수 있음

Input and Output – example

- SIC Instruction – I/O

TD	INDEV	Tests INDEV device (sets CC) < : device is ready! = : device isn't ready!
RD	INDEV	Value of A (right most byte) <- Input 1byte (INDEV)
WD	INDEV	Output 1byte (INDEV) <- Value of A (right most byte)

- RD - 디바이스에서 1byte를 읽어 A 레지스터 가장 마지막 1byte에 저장(rightmost byte)
- WD - A 레지스터의 값을 내보내는 명령
- 1바이트씩만 처리되므로, 여러바이트는 반복적으로 처리

이 동영상 교과목 자료는
GJ-RIP 사업의
지자체-대학 협력기반 지역혁신 사업비로 개발되었음.